

Student Excuse Generator – Student Excuse Prompts

Prompt 1: Core Logic

Prompt:

Create a console-based C++ program that:

- Takes user input (student name)
- Stores at least 10 excuse templates using a vector
- Uses random selection to generate a different excuse each time
- Replaces the placeholder {name} with actual user input
- Displays the final excuse clearly

Ensure:

- Code is clean and well-structured
- Variable names are meaningful
- Comments are added for each step
- Program is easy for beginners to understand



C++ Student Excuse Generator

Here's a beginner-friendly program that generates random excuses for students:



cpp



Example Output

STUDENT EXCUSE GENERATOR

Enter student name: Alex Johnson

TODAY'S EXCUSE:

Alex Johnson was abducted by aliens who needed help with their own homework

Prompt 2: Generating Excuse Templates

Prompt:

Generate at least 15 creative and realistic student excuses for missing homework.

Requirements:

- Use placeholder {name} in each sentence
- Keep excuses short, clear, and believable
- Include technical issues, personal reasons, and funny situations
- Avoid repetition
- Make them suitable for a school assignment



Here are 15 creative, short, and realistic excuses using {name}:

1. {name} had internet issues while uploading the homework.
2. {name} was feeling unwell and couldn't complete the assignment.
3. {name} accidentally left the completed homework at home.
4. {name}'s laptop crashed while finishing the last question.
5. {name} had a family emergency and couldn't focus on studies.
6. {name} misunderstood the submission date.
7. {name}'s files got deleted due to a system error.
8. {name} had a power cut during study time last night.

Prompt 3: Improvement

Prompt:
atGPT 

 Free offer 

Improve the following student excuses to make them more creative and slightly humorous while still sounding realistic.

Instructions:

- Keep placeholder {name}
- Add variety in tone and structure
- Avoid exaggeration
- Ensure excuses sound natural and relatable

Here are the excuses:

{name} tried uploading the homework, but the internet decided to stop cooperating at the worst moment.

{name} wasn't feeling well yesterday and couldn't focus properly on the assignment.

{name} completed the homework but somehow left the notebook at home in the morning rush.

{name}'s laptop froze right before saving the final answers.

{name} had a small family emergency and had to pause everything else.

{name} got confused about the submission date and realized it a bit too late.



{name}'s file disappeared after a system error—technology had other plans.

Prompt 4: Bug Fixing

Prompt:

```
return 0;  
}
```

Tasks:

- Identify and fix any bugs
- Ensure the program does not crash on invalid input
- Handle edge cases like empty name input
- Improve stability and reliability

Explain:

- What the bugs were
- How you fixed them
- Why the fixes are important

  < 2/2 >

You're already very close—this is a solid beginner program. The main issues aren't obvious "crashes," but **reliability and edge-case weaknesses**. Let's clean those up properly.

✓ Improved & Debugged Version 

Prompt 5: Improving Program Logic

Prompt:

```
return 0;  
}
```

Focus on:

- Code readability
- Efficiency
- Beginner-friendliness
- Better use of data structures
- Improving random selection logic

Also suggest:

- Additional features that can make the program more interactive
- Ways to make the program scalable

Provide improved version of the code with explanation.

You've written a solid beginner program 🍌—it works, it's readable, and the logic is clear. Now let's refine it to be **cleaner, more efficient, scalable, and closer to good C++ practices**.



 Key Issues & Improvements

Prompt 6: Edge Case Handling

Prompt:

Improve my C++ program to properly handle edge cases.

Include:

- Empty input handling
- Unexpected user input
- Prevent program crashes

Explain:

- What edge cases were handled
- Why handling edge cases is important in real-world programs



You're on the right track—your program works, but it assumes the user behaves "perfectly." Real users don't



Let's make your program **robust, safe, and beginner-friendly**.

✅ Improved C++ Program (Edge-Case Safe)



Prompt 7: Code Refactoring

Prompt

Refactor my C++ code to improve overall quality.

Requirements:

- Use meaningful variable names
- Add proper comments
- Maintain clean formatting
- Make the code beginner-friendly

Explain how these improvements make the code better.



Your program already works well—this refactor focuses on making it **cleaner, easier to read, and beginner-friendly**, while improving structure and naming.

✅ Refactored C++ Code (Clean & Beginner-Friendly)

📄 C++



```
#include <iostream>
```